Practical 7: MiniBank Runtime Metadata & Annotation Validator

Problem Definition:

Equip MiniBank with custom metadata annotations @Id, @Positive, @MaxLength, standard annotations @Override, @FunctionalInterface, and create AnnotationValidator using reflection (getDeclaredFields(), setAccessible(true), isAnnotationPresent()) to validate domain objects.

Code:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

class PartB {
    @Target(ElementType.FIELD)
    @Retention(RetentionPolicy.RUNTIME)
    public @interface Id {}

    @Target(ElementType.FIELD)
    @Retention(RetentionPolicy.RUNTIME)
    public @interface Positive {
        String message() default "must be > 0";
    }

    @Target(ElementType.FIELD)
    @Retention(RetentionPolicy.RUNTIME)
    public @interface MaxLength {
        int value();
        String message() default "exceeds maximum allowed length";
    }

    @FunctionalInterface
    public interface WithdrawRule {
        boolean allow(Account account, long amount);
    }

    public static class Account {
        @Id
        private final String accountNumber;

        @MaxLength(value = 30, message = "Holder name exceeds 30 characters")
        private final String holderName;

        @Positive(message = "must be > 0")
        private long balance;

        public Account(String accountNumber, String holderName, long balance) {
            this.accountNumber = accountNumber;
            this.holderName = holderName;
            this.balance = balance;
        }
```

```
        public String getAccountNumber() { return accountNumber; }
        public String getHolderName() { return holderName; }
        public long getBalance() { return balance; }

        @Override
        public String toString() {
            return "Account [" + accountNumber + " | " + holderName + " | Balance: Rs. " +
balance + "]";
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) return true;
            if (obj == null || getClass() != obj.getClass()) return false;
            Account acc = (Account) obj;
            return accountNumber.equals(acc.accountNumber);
        }

        @Override
        public int hashCode() {
            return accountNumber.hashCode();
        }
    }

    public static class AnnotationValidator {
        public static String[] validate(Object obj) {
            List<String> errors = new ArrayList<>();
            Class<?> clazz = obj.getClass();

            for (Field field : clazz.getDeclaredFields()) {
                field.setAccessible(true);
                try {
                    Object value = field.get(obj);

                    if (field.isAnnotationPresent(Positive.class)) {
                        Positive pos = field.getAnnotation(Positive.class);
                        if (value instanceof Number num) {
                            if (num.longValue() <= 0) {
                                errors.add("Field '" + field.getName() + "': " + pos.message() +
" (Actual: " + num + ")");
                            }
                        }
                    }

                    if (field.isAnnotationPresent(MaxLength.class)) {
                        MaxLength max = field.getAnnotation(MaxLength.class);
                        if (value != null && value.toString().length() > max.value()) {
                            errors.add("Field '" + field.getName() + "': " + max.message() + "
(Max: " + max.value() + ", Actual: " + value.toString().length() + ")");
                        }
                    }
                } catch (IllegalAccessException e) {
                    errors.add("Cannot access field " + field.getName());
                }
            }
            return errors.toArray(new String[0]);
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 7 - Part B: MiniBank Runtime Metadata & Annotation
Validator");
        System.out.println();
        Account invalidAccount = new Account("AC1001", "Rudra Modi With An Excessively Long
Account Holder Name Example", -500);
        System.out.println("Validating Problematic Account Instance:");
        System.out.println("Account Info: " + invalidAccount);
        String[] invalidErrors = AnnotationValidator.validate(invalidAccount);
        for (String err : invalidErrors) {
            System.out.println("  [VALIDATION ERROR] " + err);
        }

        System.out.println("\nValidating Compliant Account Instance:");
        Account validAccount = new Account("AC1002", "Rudra Modi", 15000);
        System.out.println("Account Info: " + validAccount);
        String[] validErrors = AnnotationValidator.validate(validAccount);
        if (validErrors.length == 0) {
            System.out.println("  [VALIDATION SUCCESS] All annotation constraints satisfied (0
errors).");
        } else {
            for (String err : validErrors) {
                System.out.println("  [VALIDATION ERROR] " + err);
            }
        }
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-7\PartB.java
RUDRA MODI || 25DCS058

Practical 7 - Part B: MiniBank Runtime Metadata & Annotation Validator

Validating Problematic Account Instance:
Account Info: Account [AC1001 | Rudra Modi With An Excessively Long Account Holder Name Example | Balance: Rs. -500]
  [VALIDATION ERROR] Field 'holderName': exceeds maximum allowed length (Max: 30, Actual: 67)
  [VALIDATION ERROR] Field 'balance': must be > 0 (Actual: -500)

Validating Compliant Account Instance:
Account Info: Account [AC1002 | Rudra Modi | Balance: Rs. 15000]
  [VALIDATION SUCCESS] All annotation constraints satisfied (0 errors).
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: How does AnnotationValidator inspect private annotated fields?

Answer: AnnotationValidator uses Java Reflection methods (getDeclaredFields(), setAccessible(true), and getAnnotation()) to inspect private domain fields and dynamically enforce constraints (@NotBlank, @MaxLength, @Positive) at runtime.

Q2: Why must custom validation annotations specify @Retention(RetentionPolicy.RUNTIME)?

Answer: RetentionPolicy.RUNTIME ensures that annotation metadata is retained in the compiled .class bytecode and remains accessible to the reflection framework during application execution.